



Snake Groove — Фаза 0: Полное итоговое резюме

Общая цель

Создание кроссплатформенной музыкальной игры **Snake Groove**, где каждая съеденная еда добавляет новый инструмент к фону, превращая классическую змейку в динамичный музыкальный опыт. Проект строится с нуля с продуманной архитектурой, разделением логики и UI, и подготовкой к CI/CD и публикации.

Установлено и настроено

Среда и инструменты

- **Windows 11 Pro** (основная рабочая ОС)
 - **Unity Hub + Unity 6.2 (6000.2.4f1)** с установленными модулями:
 - Windows Build Support (IL2CPP)
 - WebGL Build Support
 - Android Build Support (с SDK, NDK, OpenJDK)
 - URP (Universal Render Pipeline)
 - **Visual Studio 2022 Community** + плагин **Unity** и **GitHub Copilot** (авторизация через GitHub)
 - **Git 2.49.1** + **Git LFS 3.7.0** (установлены и проверены)
 - **GitHub Desktop** (репозиторий синхронизирован)
 - **OneDrive** (используется только для резервных копий)
-

Создано

Репозиторий GitHub

- Название: **snake-groove**
- Лицензия: **MIT**
- Структура папок:

```
snake-groove/  
  SnakeGroove/           # Unity проект  
  .gitattributes  
  .gitignore  
  LICENSE  
  README.md
```

- Включен **Git LFS** для хранения тяжёлых ассетов (png, jpg, wav, mp3, fbx, blend и др.)

Unity проект

- Проект создан на **URP 2D Template** (Universal 2D Render Pipeline)
- Имя проекта: **SnakeGroove**
- Базовая структура `Assets/`:

```
Assets/  
  Art/  
  Audio/  
  Prefabs/  
  Scenes/           # Boot, MainMenu, Game  
  Scripts/  
    Core/           # GameManager, EventBus, BootLoader  
    Infrastructure/ # адаптеры, DI, сервисы  
    UI/             # интерфейсы, меню, HUD  
    Gameplay/       # логика змейки, поле, еда  
  Settings/  
  Tests/
```

Сцены

- **Boot.unity** — точка входа, вызывает загрузчик сцен
- **MainMenu.unity** — меню (пока тестовый текст TMP)
- **Game.unity** — игровая сцена (пустая, добавляется SnakeRunner позже)

План с текущего момента

0. Скрипты ядра Unity-уровня (Core)

- **GameManager.cs** — Singleton с DontDestroyOnLoad
- **EventBus.cs** — простая система событий (Observer)
- **BootLoader.cs** — загрузка сцены `MainMenu` после Boot

Проверить: сцены переходят корректно, `GameManager` живёт между сценами.

A. Завершение каркаса и тест перехода сцен

1. Проверить работу `BootLoader` → переход в `MainMenu`.
2. Добавить в `MainMenu` визуальный элемент (текст или кнопку).
3. Убедиться, что `GameManager` не пересоздаётся при смене сцен.
4. Сделать коммит: `feat(core): add GameManager, EventBus, BootLoader and scene jump Boot->MainMenu`

B. Подготовка к разделению логики

1. Создать embedded-package для ядра (вне Unity-папки):

```
snake-groove/  
  Packages/
```

```
com.jochen.snakecore/  
package.json  
Runtime/  
Snake.Core.asmdef
```

2. Настроить `package.json` и `asmdef` для ядра.
3. Перезапустить Unity — пакет появится в **Package Manager**.

C. Создание чистого ядра (pure C#)

В `Packages/com.jochen.snakecore/Runtime/`:

- **Direction.cs** — enum направлений
- **GridPos.cs** — структура координат сетки
- **Contracts.cs** — интерфейсы `ISnakeGame`, `IRandom`
- **SnakeGame.cs** — основная игровая логика (без Unity)

Коммит: `feat(core): add embedded package 'Snake Core' with pure C# game logic`

D. Интеграция Unity и ядра

1. В `Assets/Scripts/Infrastructure/Unity/` создать `Snake.Unity.asmdef` и добавить reference на `Snake.Core`.
2. Создать адаптер `SnakeRunner.cs`, который:
3. принимает ввод (WASD/стрелки);
4. обновляет ядро по таймеру (`ticksPerSecond`);
5. обрабатывает Game Over через исключение `SnakeGameOverException`.
6. Подключить `SnakeRunner` в сцене `Game`.
7. Проверить — при запуске и движении WASD ядро тикает, при столкновении выводит `Game Over` в консоль.

Коммит: `feat(infra): add Unity adapter (SnakeRunner) and wire to Snake.Core`

Дальнейшие шаги

- Добавить визуализацию змейки (спрайты, Tilemap, Pixel Perfect Camera)
- Добавить музыкальные слои и систему звуковых событий
- Настроить CI/CD через GitHub Actions и Unity Cloud Build
- Подготовить страницу проекта (README, скриншоты, roadmap)

Ключевые понятия и подходы

- **Отделение ядра** — чистый C#, без UnityEngine (подходит для WebGL, Android, iOS, Windows).
- **UI/рендер** — на уровне Unity, использует данные ядра.
- **Design Patterns:**
 - Singleton (GameManager)
 - Observer (EventBus)

- Factory (объекты еды и эффектов)
 - Strategy (разные типы еды/поведения)
 - **Качество кода:** Clean Code, короткие функции, осмысленные имена, комментарии для Copilot.
-

Мотивация

Ты уже выстроил прочную основу: движок, система контроля версий, CI/CD, архитектура и структура проекта. Следующий шаг — оживить змейку, сделать музыку частью игрового процесса и постепенно превратить это в масштабируемую игровую платформу.